

Redis整理

一、资料

1、Redis数据持久化机制

- https://blog.csdn.net/m0_63947499/article/details/143026087
- <https://open8gu.com/redis/persistent/oegct3ayo729baqc/>

2、Redis数据类型详解

1.五种数据类型：字符串（String）、哈希（map）、列表（List）、集合（Set集合）、有序结合（Sorted Set 即Zset有序集合）

- [基础操作命令](#)
- [基础操作&使用场景](#)

2.四种特殊的数据结构：位图（Bitmap）、HyperLogLog(基数统计),Geospatial(地理信息),Stream

- https://blog.csdn.net/qg_73924465/article/details/141504379
- [位图详解](#)

3.各个数据类型的底层实现

字符串：int（8个字节的长整型）、SDS

哈希：zipList、Hashtable(数组+列表)

列表：LinkedList+zipList（低版本）、quickList+zipList（高版本）

集合：

有序集合：

- https://blog.csdn.net/weixin_45690643/article/details/135890450
- <https://pdai.tech/md/db/nosql-redis/db-redis-x-redis-object.html>
- 跳表详解：<https://blog.csdn.net/sssxlxbwz/article/details/123769262>

3、Redis为什么快？

参考文章：

- https://blog.csdn.net/qg_38196449/article/details/144583793
- <https://open8gu.com/redis/trending/utx7gm7khpufug3h6/>
- <https://www.cnblogs.com/coderacademy/p/18099027> (还没看)
- <https://juejin.cn/post/6978280894704386079> (还没看)

Redis 官方早前发布过一套基准测试，在 Redis 服务连接数小于 1 万时，并发数量每秒可以达到 10-12 万左右。连接数在 3-6 万 时，也能支持每秒 5-6 万的并发。

- **从存储方式上看：**Redis 是基于内存的数据库（当然也支持AOF和RDB的持久化操作），而直接访问内存的速度要比访问磁盘高上几个数量级。这是 Redis 快最主要的原因。

磁盘内存横向对比数据的佐证：计算机访问一次 SSD 硬盘大概需要 50 ~ 150 微秒，如果是传统的硬盘时间则需要 1 ~ 10 毫秒，而访问一次内存仅需要 120 纳秒。可见，磁盘和内存访问的速度差了数个数量级。

注意：如果涉及到AOF和RDB持久化的，需要和磁盘打交道：除了少数一些需要跟磁盘打交道的时候（比如持久化），大部分时候 Redis 都只在内存中进行读写，因此它的效率极高。

- **从线程模型上看：**Redis 使用单线程模型，这意味着它的所有操作都是在一个线程内完成的，不需要进行线程切换和上下文切换（操作系统需要从一个线程切换到另一个线程，保存和恢复相应的状态）。这大大提高了 Redis 的运行效率和响应速度

其核心思想是通过事件循环和 I/O 多路复用（如 `epoll` 或 `select`）来处理所有客户端的请求。由于每个请求都在同一个线程中顺序执行，避免了多线程模型中常见的线程切换、同步和上下文切换的开销，以及多线程带来的线程安全问题。**虽然 Redis 的单个主线程模型确实带来的不少的好处，但是这个设计更多的还是在性能与设计之间（即与硬件有密切关系）取得的一个平衡。**实际上不少市面上开源或者大公司内部自研的 KV 数据库——比如 KeyDB 或者 Dragonfly——都是基于多线程模型实现的，它们以单机模式运行在多核机器上时也确实表现出了比 Redis 更高的性能，因为他们能更好地去利用CPU的多核进行操作。总的来说，Redis的单线程更加适用于**I/O 密集型的任务**，Redis 在 I/O 密集型任务（如简单的数据访问、缓存等）上表现很好，但如果应用涉及到复杂的计算或者需要大量并发的读写，单线程就会成为瓶颈。KeyDB 和 Dragonfly 通过多线程使得 CPU 密集型任务也能够多个核上并行处理，从而提供更高的吞吐量和响应速度，更加适用CPU密集型任务。

注意：不过，Redis 也并非真的就是单线程的，从 4.0 开始，Redis 就引入了 `UNLINK` 这类命令，用于异步执行删除等重操作，并在 6.0 以后引入了专门的 IO 线程，实现了多线程的非阻塞式 IO，它们也进一步的提升了 Redis 的执行效率。

- **从IO模型上看：**Redis 在单线程的基础上，采用了 I/O 多路复用技术，实现了单个线程同时处理多个客户端连接的能力，从而提高了 Redis 的并发性能

关于各个操作系统的I/O复用机制：

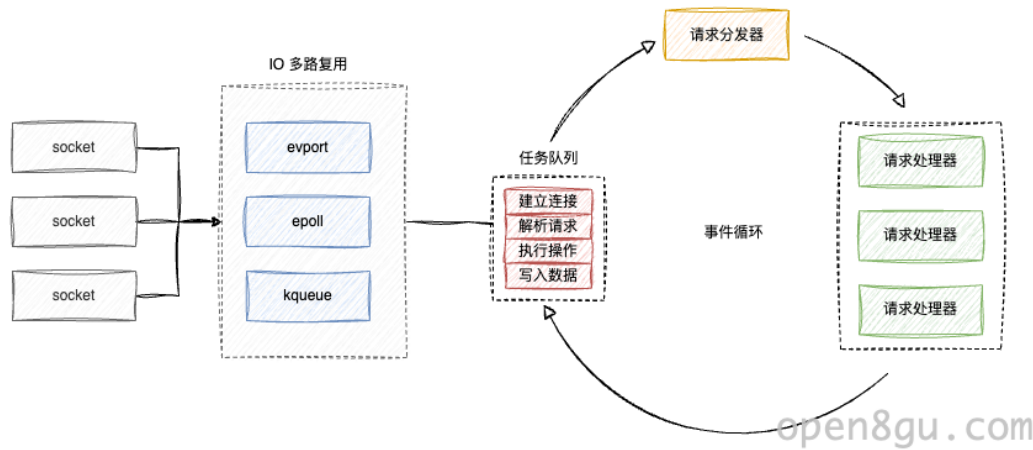
`evport` (Solaris)、`epoll` (Linux)、和 `kqueue` (Mac OS/FreeBSD) 都是操作系统提供的高效 I/O 多路复用机制（非IO阻塞）。它们的主要作用是提供一种方式，使得程序可以高效地监视多个 I/O 流（例如，文件描述符、套接字等），并在有数据准备好或事件发生时进行处理。它们广泛用于需要处理大量并发 I/O 请求的高性能服务器或网络应用。

I/O复用机制的目的：

1. **高效的 I/O 多路复用：**这些机制的基本目的是让单一线程能够高效地监视多个 I/O 通道，而不需要为每个 I/O 通道创建单独的线程或进程。它们允许程序在某一时刻等待多个 I/O 操作的完成，从而在 I/O 阻塞时不浪费系统资源。
2. **避免阻塞：**在传统的阻塞 I/O 模型中，程序会在 I/O 操作完成之前一直等待，这样会浪费 CPU 资源。而这些高效 I/O 多路复用机制允许程序在等待 I/O 操作时可以去执行其他任务，从而提高了应用程序的响应能力。

在这个模型中，它将会来自客户端的网络请求作为一个事件发布到队列中，然后线程将同步的获取事件并派发到不同的处理器，而处理器处理完毕后会再发布另一个事件.....整个主流程都由 Redis 的主线程在一个不间断的循环中完成，这就是事件循环。

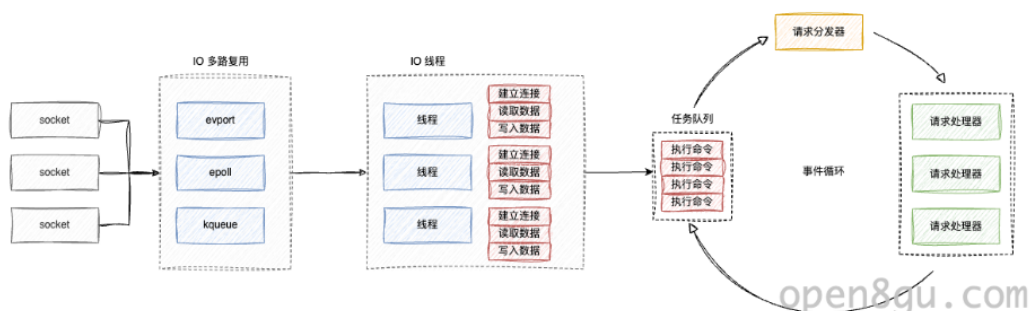
熟悉 Netty 的同学可能会觉得有点既视感，因为两者都可以认为基于反应器模式实现的 IO 模型，不过 Netty 可以有多个事件循环，并且还可以划分为 Boss 和 Worker 两类事件循环组，而 Redis 只有一个事件循环，并且在早期版本只有一个 IO 线程（也就是主线程本身）。



- **从基本数据的数据结构上看：** Redis 提供了多种高效的数据结构，如哈希表、有序集合、列表等，这些数据结构都被实现得非常高效，能够在 $O(1)$ 的时间复杂度内完成数据读写操作，这也是 Redis 能够快速处理数据请求的重要因素之一（数据结构要熟悉，参考2、Redis数据类型详解）
- **6.0版本多线程的引入：** 在 Redis 6.0 中，为了进一步提升 IO 的性能，引入了多线程的机制。采用多线程，使得网络处理的请求并发进行，就可以大大的提升性能。多线程除了可以减少由于网络 I/O 等待造成的影响，还可以充分利用 CPU 的多核优势。

随着请求规模的扩大，单个线程在网络 IO 上消耗的 CPU 时间越来越多，它逐渐成为了 Redis 的性能瓶颈。因此在 6.0 版及以上版本，Redis 正式引入了多线程来处理网络 IO。

在新的版本中，Redis 依然使用单个主线程来执行命令，但是使用多个线程来处理 IO 请求，主线程不再负责包括建立连接、读取数据和回写数据这些事情，而只是专注于执行命令。



这个做法在保证单线程设计的原有优点的情况下，又进一步提高了网络 IO 的处理效率。

4、为什么设计为单线程？

参考文章：https://blog.csdn.net/qq_38196449/article/details/144583793

Redis 作为一个成熟的分布式缓存框架，它由很多个模块组成，如网络请求模块、索引模块、存储模块、高可用集群支撑模块、数据操作模块等。

我们所说的 Redis 单线程，指的是“其网络 IO 和键值对读写是由一个线程完成的”，也就是说，Redis 中只有网络请求模块和数据操作模块是单线程的。而其他的如持久化存储模块、集群支撑模块等是多线程的。所以说，Redis 中并不是没有多线程模型的，早在 Redis 4.0 的时候就已经针对部分命令做了多线程化。

那么，为什么网络操作模块和数据存储模块最初并没有使用多线程呢？

一个计算机程序在执行的过程中，主要需要进行两种操作分别是读写操作（I/O操作，其中包括网络 I/O 和磁盘 I/O）和计算操作（设计CPU资源，消耗CPU资源）。

多线程的目的，就是通过并发的方式来提升 I/O 的利用率和 CPU 的利用率。那么，Redis 需不需要通过多线程的方式来提升提升 I/O 的利用率和 CPU 的利用率呢？我们可以肯定地说，Redis 不需要提升 CPU 利用率，因为 Redis 的操作基本都是基于内存的，CPU 资源根本就不是 Redis 的性能瓶颈，在 Redis 的场景下，由于数据存储在内存中，内存的访问速度比磁盘快得多，这意味着 CPU 在访问内存中的数据时非常迅速，且消耗较少的时间。因此，Redis 以非常高的速度处理内存中的数据，而不需要大量依赖 CPU 来进行计算或复杂的操作。因此，通过多线程技术来提升 Redis 的 CPU 利用率这一点是完全没必要的。Redis 确实是一个 I/O 操作密集的框架，他的数据操作过程中，会有大量的网络 I/O 和磁盘 I/O 的发生。要想提升 Redis 的性能，是一定要提升 Redis 的 I/O 利用率的，这一点毋庸置疑。但是，**提升 I/O 利用率，并不是只有采用多线程技术这一条路可以走！**

多线程的引入有很多弊端，如线程安全问题、线程切换、上下文切换带来的开销，会为语言和框架带来了更多的复杂性，所以，在提升 I/O 利用率这个方面上，Redis 并没有采用多线程技术，而是选择了多路复用 I/O 技术。

总结：

Redis 并没有在网络请求模块和数据操作模块中使用多线程模型，主要是基于以下四个原因：

- Redis 操作基于内存，绝大多数操作的性能瓶颈不在 CPU。
- 使用单线程模型，可维护性更高，开发、调试和维护的成本更低。
- 单线程模型，避免了线程间切换带来的性能开销。
- 在单线程中使用多路复用 I/O 技术也能提升 Redis 的 I/O 利用率。

还是要记住：Redis 并不是完全单线程的，只是有关键的键值对读写是由一个线程完成的。

5、为什么6.0之后又引入多线程？

https://blog.csdn.net/qg_38196449/article/details/141391514

6、SpringBoot集成Redis方式

https://blog.csdn.net/qg_20236937/article/details/137561788

7、Redis删除过期策略

https://blog.csdn.net/qg_33204709/article/details/139513833

<https://open8gu.com/redis/trending/fye8babys322xmsm/>

1. 背景

我们都知道 Redis 是一种内存数据，所有的数据均存储在内存中，可以通过 ttl 指令查看数据的状态：

- xx：表示具有时效性的数据。
- -1：表示永久性的数据。
- -2：表示过期的数据，或已经删除的数据，或未定义的数据。

但是 过期的数据真的被删除了吗？

- 在 Redis 的执行过程中，会让 CPU 处理很多的指令，CPU 可能会一下子处理不过来，这样就需要区分优先级了。
- 很明显 Redis 的 key 过期删除操作没有那么重要，所以就先不删除，继续保存在内存中。
- 那什么时候删除呢？这就是 Redis 的删除策略做的事了。

删除策略的目标：就是在内存和 CPU 占用之间找到一个平衡，在 CPU 空闲的时候再去删除过期的数据，防止出现 CPU 过载导致服务器宕机、内存泄漏（CPU 过载，导致无法进行过期数据的删除，导致内存中存在大量的过期数据，出现内存泄漏）等问题。

Redis 采用的是 定期删除 + 惰性删除 的策略。

2. 三种删除策略

1 定时删除（用 CPU 换内存空间）

实现方式：创建一个定时器，每隔一段时间，定时器会对全量 key 的扫描，并删除过期的 key。

- 优点：节约内存，可以定期立即释放掉所有不必要的内存空间。
- 缺点：CPU 的压力会很大，不会考虑删除的时候 CPU 是否空闲，会影响 Redis 服务器的响应时间和吞吐量。
- 应用场景：适用于对数据过期时间要求不高，数据量较小的场景。

2 定期删除

实现方式：由 redis.c 的 `activeExpireCycle()` 函数实现，周期性轮询 Redis 库中的时效性数据，采用随机抽取策略，利用过期数据占比的方式控制删除的频率。

Redis 启动服务器初始化时，读取配置 `server.hz` 的值，默认为 10。

每秒钟执行 `server.hz` 次 `serverCron()` -> `databaseCron()` -> `activeExpireCycle()`。

`activeExpireCycle()` 对每个 `expires[]` 逐一进行检测，每次执行 $250\text{ms}/\text{server.hz}$ （没有执行完，会去下一个周期进行处理）。

对某个 `expires[]` 检测时，随机挑选 W 个 key 检测：

如果 key 超时，删除 key；

如果一轮中删除的 key 的数量 $> W * 25\%$ ，循环该过程；

如果一轮中删除的 key 的数量 $\leq W * 25\%$ ，检查下一个 `expires[]`，0~15 循环；

W 取值 = `ACTIVE_EXPIRE_CYCLE_LOOKUPS_PER_LOOP` 属性值。

参数 `current_db` 用于记录 `activeExpireCycle()` 进入了哪个 `expires[]` 执行。

如果 `activeExpireCycle()` 执行时间到期，下次从 `current_db` 继续向下执行。

- 优点：CPU 性能占用设置有峰值，检测频率可自定义设置，相对于定时删除，可以更灵活地控制 CPU 占用，适用于数据量较大的情况。内存压力不是很大，长期占用内存的冷数据会被持续清理。
- 缺点：可能会导致短时间内部分内存无法及时释放，对系统性能有一定影响。
- 应用场景：适用于数据量较大、对内存占用没有特别要求的场景。

补充：

1、定期删除函数的运行频率，在 Redis 2.6 版本中，规定每秒运行 10 次，大概 100ms 运行一次。在 Redis 2.8 版本后，可以通过修改配置文件 `redis.conf` 的 `hz` 选项来调整这个次数。

2、关于 `expires[*]`

假设我们设置了一个过期的键值对，过期时间为 5 秒，并且该键的哈希槽为 10。我们来具体描述一下 `expires[*]` 如何运作。

假设：

- Redis 中有多个数据库（假设数据库 0），每个数据库都有一个 `expires[]` 数组。
- 假设 Redis 默认配置有 16 个数据库（0 到 15），而我们在数据库 0 中操作。
- 我们设置一个键的过期时间为 5 秒。

- 该键的哈希槽为10。

具体步骤：

1. 设置键值对：

- 假设你设置了一个键值对 `SET mykey value`，并且指定了过期时间为5秒。你执行的是 `SET mykey value EX 5`（5秒后过期）。
- Redis 会将这个键 `mykey` 存储在数据库 0 的哈希槽为 10 的位置，并将其过期时间设置为当前时间 + 5秒。

2. `expires[]` 数组：

- Redis 会为数据库 0 创建一个 `expires[]` 数组，这个数组存储了每个键对应的过期时间戳（或者是 `NULL`，表示没有过期时间）。
- 假设 `mykey` 在哈希槽 10，所以 `expires[10]` 就会被设置为过期时间戳，即当前时间戳加上 5秒。

例如，假设当前时间戳是 `t`，那么 `expires[10]` 会被设置为 `t + 5`。

3. 过期时间的检查：

- 在 Redis 内部，每次执行 **惰性删除** 或 **定期删除** 时，Redis 会检查 `expires[]` 数组中的所有键。
- 在这次扫描时，Redis 会查看哈希槽 10 对应的 `expires[10]`，检查该时间是否已经超过了当前时间。如果 `expires[10]` 小于当前时间，那么这个键 `mykey` 就已经过期，Redis 会删除它。

4. `expires[]` 数组示例：假设 `expires[]` 数组的结构如下：

```
textCopy Codeexpires[0] = NULL
expires[1] = NULL
expires[2] = NULL
...
expires[9] = NULL
expires[10] = t + 5 （即 5秒后的时间戳）
expires[11] = NULL
...
expires[15] = NULL
```

在上述示例中，`expires[10]` 记录了键 `mykey` 的过期时间戳。如果在 `t + 5` 秒时，Redis 会检查 `expires[10]`，发现该时间戳已经过去，Redis 就会删除该键。

5. 删除过期键：

- 在 Redis 的 **定期删除** 机制中，`activeExpireCycle` 会定期扫描这个 `expires[]` 数组，检查哪些键已经过期，并将它们从数据库中删除。
- 在扫描 `expires[]` 数组时，Redis 会注意到 `expires[10]` 的时间戳已经超过当前时间，表示 `mykey` 已经过期，因此会删除该键。

总结：

在 Redis 中，`expires[]` 数组用于存储每个键的过期时间戳。如果某个键的哈希槽为10，Redis 会将该键的过期时间记录在 `expires[10]` 中。当过期时间到达时，Redis 会删除该键并清理内存。这是 Redis 内部用来实现键的过期和内存管理的机制。

3 惰性删除（用内存换CPU性能）

实现方式：数据到达过期时间，不做处理。等下次访问该数据时，发现未过期，则返回值，发现已经过期，删除 expires 空间和 key 值，并返回不存在。

- 优点：节约 CPU 性能，当数据必须删除的时候才删除。
- 缺点：内存压力较大，会出现过期数据长期占用内存的情况。
- 应用场景：适用于对数据过期时间要求不严格、对内存占用要求宽松的场景

8、内存淘汰策略

与到期删除策略不同，内存淘汰策略主要目的则是为了防止运行时内存超过最大内存，所以尽管最终目的都是清理内存中的一些 Key，但是它们的应用场景和触发时机是不同的。

<https://open8gu.com/redis/trending/drx30l2p8a6klao5/>

淘汰过程：

1. 每次当 Redis 执行命令时，若设置了最大内存大小 `maxmemory`，并设置了淘汰策略式，则会尝试进行一次 Key 淘汰；
2. Redis 首先会评估已使用内存（这里不包含主从复制使用的两个缓冲区占用的内存）是否大于 `maxmemory`，如果没有则直接返回，否则将计算当前需要释放多少内存，随后开始根据策略淘汰符合条件的 Key；当开始进行淘汰时，将会依次对每个数据库进行抽样，抽样的数据范围由策略决定，而样本数量则由 `maxmemory-samples` 配置决定；
3. 完成抽样后，Redis 会尝试将样本放入提前初始化好 `EvictionPoolLRU` 数组中，它相当于一个临时缓冲区，当数组填满以后即将里面全部的 Key 进行删除。
4. 若一次删除后内存仍然不足，则再次重复上一步骤，将样本中的剩余 Key 再次填入数组中进行删除，直到释放了足够的内存，或者本次抽样的所有 Key 都被删除完毕（如果此时内存还是不足，那么就重新执行一次淘汰流程）。

抽样处理：

在抽样这一步，涉及到从字典中随机抽样这个过程，由于哈希表的 Key 是散列分布的，因此会有很多桶都是空的，纯随机效率可能会很低。因此，Redis 采用了一个特别的做法，那就是先连续遍历数个桶，如果都是空的，再随机调到另一个位置，再连续遍历几个桶.....如此循环，直到结束抽样。

9、Redis是不是CPU核数越高越好

<https://open8gu.com/redis/trending/rp5mw9n3c6pdla7a/>

可以讲讲为什么一开始Redis要设置成单线程，为什么快，因为其是IO密集型，与CPU核数关系不大，并且解决IO密集型的方式不一定要依赖多线程的方式，但一定要讲出来，现在6.0之后也引入了多线程的方式！

Redis 是单线程的，这意味着在任何给定时刻只能处理一个请求(但是现在6.0之后，也引入了多线程模式，处理IO都是多线程去处理的，可以处理多个请求)

其是IO密集型，而不是CPU密集型，没有很多的计算操作，只是key-value的存取。多核CPU核心可以帮助Redis 更好地处理并发请求，而IO密集型的解决方案一开始使用的就是IO多路复用，而没有用到多线程，也是为了避免多线程带来的线程上下文切换、线程安全等带来的问题和复杂。

因此，它更侧重于单个核心的性能，而不是多核心。高速缓存访问和处理请求的速度可能会受到 CPU 速度的限制，而不是核心数。

当然，不能真的为 Redis 只分配 1 个核心，因为 Redis 除了主线程处理从客户端发起的读写请求外，还会有一些异步的处理，比如：持久化操作、主从复制等，所以推荐设置 2 核 CPU 即可。

我看了腾讯云 Redis 默认会为每个节点分配 2 核 CPU，1 个 CPU 负责主线程处理读写请求，另外 1 个 CPU 用于处理后台任务。

10、提升Redis的批量访问性能

<https://open8gu.com/redis/trending/cu5omslxak0qb98p/>

一个是从 API 操作命令上优化，另一个则是通过聚合批命令节省网络 IO

- 批量命令（通过批量API实现优化）

以 Redis Hash 结构举例，涉及到批量操作，Redis 提供了 `MGET` 和 `MSET` 命令，可以一次性获取多个键的值或者设置多个键的值。

- Pipeline 管道（聚合批量命令）

Redis Pipeline 是提高 Redis 性能的一个非常有效的方式，特别是在处理多个命令时。通过将多个命令组合在一起，可以减少网络延迟和提高系统吞吐量。在不同编程语言中，Redis 都提供了相应的客户端库支持 Pipeline 操作，能够让你更高效地进行批量操作。

使用 Pipeline 可以在一次网络往返中发送多个命令，从而减少了通信开销。通过将多个命令打包发送，可以极大地提升批量访问的性能。使用上需要注意：Pipeline 并不会保证原子性，所以在涉及事务和多个命令的情况下，需要格外小心。

- LUA 脚本（聚合批量命令）

如果需要保证原子性的同时还需要批量特性，可以使用 Lua 脚本在 Redis 机器上原子地执行多个命令序列。这对于需要在多个键之间执行复杂操作的情况非常有用。

Pipeline操作实现（客户端使用jedis）：

```
import redis.clients.jedis.Jedis;
import redis.clients.jedis.Pipeline;

import java.util.List;

public class RedisPipelineExample {
    public static void main(String[] args) {
        // 创建 Jedis 连接
        Jedis jedis = new Jedis("localhost", 6379); // 假设 Redis 运行在 localhost:6379

        // 使用 Pipeline
        Pipeline pipeline = jedis.pipelined();

        // 批量设置键值对
        pipeline.set("key1", "value1");
        pipeline.set("key2", "value2");
        pipeline.set("key3", "value3");
```



```

// 批量获取值
pipeline.get("key1");
pipeline.get("key2");
pipeline.get("key3");

// 执行管道中的所有命令
List<Object> responses = pipeline.syncAndReturnAll(); // 返回所有命令的结果

// 打印响应结果
for (Object response : responses) {
    System.out.println(response);
}

// 关闭 Jedis 连接
jedis.close();
}
}

```

lua脚本执行（客户端使用jedis）：

```

import redis.clients.jedis.Jedis;

public class RedisLuaScriptExample {
    public static void main(String[] args) {
        // 创建 Jedis 连接
        Jedis jedis = new Jedis("localhost", 6379);

        // Lua 脚本：设置一个键值对并返回设置的值
        String luaScript = "redis.call('set', KEYS[1], ARGV[1]) return\nredis.call('get', KEYS[1])";

        // 执行 Lua 脚本
        String result = (String) jedis.eval(luaScript, 1, "myKey", "myValue");

        // 输出结果
        System.out.println("Lua script result: " + result);

        // 关闭连接
        jedis.close();
    }
}

```

12、Redis消息传递——发布订阅模式&实现

<https://pdai.tech/md/db/nosql-redis/db-redis-x-pub-sub.html>

Redis有两种发布/订阅模式：

- 基于频道(Channel)的发布/订阅
- 基于模式(pattern)的发布/订阅

1、基于频道(Channel)的发布/订阅

尝试：开三个客户端，一个作为发布者，另外两个作为订阅者

订阅者1：

```
127.0.0.1:6379> subscribe channel:1
1) "subscribe"
2) "channel:1"
3) (integer) 1
Reading messages... (press Ctrl-C to quit or any other key)
```

订阅者2：

```
127.0.0.1:6379> subscribe channel:1
1) "subscribe"
2) "channel:1"
3) (integer) 1
Reading messages... (press Ctrl-C to quit or any other key)
```

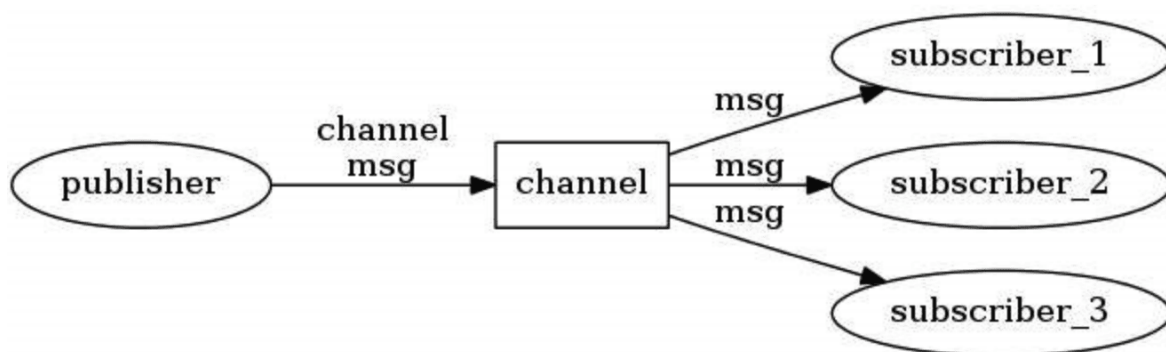
发布者：发布消息

```
127.0.0.1:6379> publish channel:1 miao
(integer) 2
```

订阅者收到消息

```
127.0.0.1:6379> subscribe channel:1
1) "subscribe"
2) "channel:1"
3) (integer) 1
1) "message"
2) "channel:1"
3) "miao"
Reading messages... (press Ctrl-C to quit or any other key)
```

图示：



- 发布者发布消息

发布者发布消息的命令是 `publish`,用法是 `publish channel message`，如向 `channel:1`说一声hi

```
127.0.0.1:6379> publish channel:1 hi
(integer) 1
```

这样消息就发出去了。返回值表示接收这条消息的订阅者数量。发出去的消息不会被持久化，也就是有客户端订阅`channel:1`后只能接收到后续发布到该频道的消息，之前的就接收不到了。

- **订阅者订阅频道**

订阅频道的命令是 `subscribe`，可以同时订阅多个频道，用法是 `subscribe channel1 [channel2 ...]`，例如新开一个客户端订阅上面频道:(不会收到消息，因为不会收到订阅之前就发布到该频道的消息)

```
127.0.0.1:6379> subscribe channel:1
Reading messages... (press Ctrl-C to quit)
1) "subscribe" // 消息类型
2) "channel:1" // 频道
3) "hi" // 消息内容
```

执行上面命令客户端会进入订阅状态，处于此状态下客户端不能使用除 `subscribe`、`unsubscribe`、`psubscribe` 和 `punsubscribe` 这四个属于“发布/订阅”之外的命令，否则会报错。

进入订阅状态后客户端可能收到3种类型的回复。每种类型的回复都包含3个值，第一个值是消息的类型，根据消息类型的不同，第二个和第三个参数的含义可能不同。

消息类型的取值可能是以下3个:

- `subscribe`。表示订阅成功的反馈信息。第二个值是订阅成功的频道名称，第三个是当前客户端订阅的频道数量。
- `message`。表示接收到的消息，第二个值表示产生消息的频道名称，第三个值是消息的内容。
- `unsubscribe`。表示成功取消订阅某个频道。第二个值是对应的频道名称，第三个值是当前客户端订阅的频道数量，当此值为0时客户端会退出订阅状态，之后就可以执行其他非“发布/订阅”模式的命令了。

2、基于模式(pattern)的发布/订阅

如果有某个/某些模式和这个频道匹配的话，那么所有订阅这个/这些频道的客户端也同样会收到信息。

.....

3、基于频道(Channel)的发布/订阅实现

.....

4、基于模式(pattern)的发布/订阅的实现

.....

5、Spring项目发布/订阅的应用

定义redis监听器

```
@Slf4j
@Configuration
public class RedisConfig {

    // redis监听配置
    @Bean
    public RedisMessageListenerContainer redisContainer(RedisConnectionFactory
redisConnectionFactory) {
        RedisMessageListenerContainer container = new
RedisMessageListenerContainer();
        container.setConnectionFactory(redisConnectionFactory);
        container.addListener((message, bytes) -> {
```

```

        log.warn("接收到重新JVM 重新加载路由事件");
        System.out.println("我收到了! , 消息内容: " + message);
    }, new ChannelTopic("channel"));
    return container;
}

}

```

定义发布者

```

@RestController
@RequestMapping("/demo")
public class democontroller {

    @Resource
    private RedisTemplate redisTemplate;

    @GetMapping
    public void convertAndSend() {
        redisTemplate.convertAndSend("channel", "nihao!Hi!");
    }
}

```

调用接口:

```

2025-02-13 15:20:58.437 WARN 11700 --- [edisContainer-2] com.example.config.Red:
我收到了! , 消息内容: ��t  nihao!Hi!

```

乱码问题:

乱码通常是由于消息的编码方式不一致导致的。在你的代码中, 可能是因为 **消息的发送方** (`redisTemplate.convertAndSend`) 和 **接收方** (`RedisMessageListenerContainer`) 的编码设置不一致, 导致了消息在传输过程中被解析为乱码。

在 `convertAndSend` 方法中发送了字符串 `"nihao!Hi!"`, 理论上它应该被正确编码为字节流。可以检查一下 `redisTemplate` 的序列化器, 确保它正确使用了 `UTF-8` 编码。

可以在 `redisTemplate` 中设置编码方式, 例如

```

@Bean
public RedisTemplate<String, String> redisTemplate(RedisConnectionFactory
connectionFactory) {
    RedisTemplate<String, String> template = new RedisTemplate<>();
    template.setConnectionFactory(connectionFactory);

    // 设置使用 String 类型的序列化器, 确保发送的是字符串
    template.setDefaultSerializer(new StringRedisSerializer());

    return template;
}

```

`StringRedisSerializer` 会将 `String` 编码为 UTF-8 字节流，从而保证消息在发送和接收时的编码一致。

注意：如果没有显式配置序列化器，它可能会使用默认的序列化器（例如，`JdkSerializationRedisSerializer`），这会导致对象以二进制格式发送，从而产生乱码。

具体的应用可以看下PIGX框架，对动态路由中对redis的应用。

13、Redis事件机制

- <https://developer.aliyun.com/article/1265747>
- <https://blog.junphp.com/details/634.jsp>
- <https://www.cnblogs.com/jiujuan/p/16723573.html>
- <https://pdai.tech/md/db/nosql-redis/db-redis-x-event.html>

问题：

1、libevent或libev 都是IO多路复用的解决方案吗

是的，`libevent` 和 `libev` 都是用于实现 **IO多路复用** 的库，它们提供了一种高效的机制来处理多个文件描述符上的输入/输出事件。具体来说，它们用于在单个线程中处理多个网络连接或文件描述符上的事件，而不需要为每个连接或描述符创建一个独立的线程或进程。

1. 事件驱动模型：

- 当客户端连接到 Redis 时，操作系统会为每个连接分配一个 **文件描述符 (FD)**。Redis 使用 `select` 或 `epoll` 等 I/O 多路复用技术来监控多个文件描述符。
- `select` 或 `epoll` 会检查每个文件描述符（即客户端连接）是否有数据可读或可写（比如客户端发送了请求，或者需要返回响应）。当发现某个文件描述符有请求时，它会将该请求加入一个**事件队列**。

2. 事件队列处理：

- Redis 通过 **事件循环**（Event Loop）来持续监控这个事件队列，并逐一处理队列中的事件。
- 事件队列中的每个事件通常是由网络事件触发的（如客户端的请求）。这些事件会被 Redis 处理函数依次取出，Redis 会根据请求类型（如 `GET`、`SET` 等）执行相应的操作。

3. 单线程的特点：

- 由于 Redis 采用单线程模型，只有一个主线程在处理所有的请求，这个主线程会不断从事件队列中取出事件进行处理。每次取出一个事件时，主线程都会执行相应的处理逻辑，然后继续处理下一个事件。

- 所以，Redis 并不是一直在处理所有事件，而是当事件队列中有事件时，主线程就会从队列中取出事件进行处理。处理完一个事件后，Redis 会接着处理下一个事件。

4. 为什么是“不断处理”：

- 这里的“不断处理”指的是，当事件队列中有待处理的事件时，Redis 会持续处理它们。通常情况下，Redis 在整个运行过程中会不断接收客户端请求，并将这些请求对应的事件加入到事件队列中，而 Redis 的主线程则不断地从队列中取出这些事件并处理。
- 但是，如果事件队列为空，Redis 主线程会暂停处理，直到新的事件到达。这种“处理”是基于事件的回调机制，因此 Redis 在实际运行时并不是持续不断地处理，而是根据是否有事件到达来控制工作。

14、redis事务

<https://pdai.tech/md/db/nosql-redis/db-redis-x-trans.html>

说明：

1、redis的事务不存在回滚的说法，如果是在multi下的命令中，出现了编译错误，当exec的时候，就会不执行该事务中的所有命令（提示：(error) EXECABORT Transaction discarded because of previous errors.）。如果是运行时的异常，在exec会跳过有异常的命令，其他命令正常执行。

2、watch xx 开启了对xx的监控，之后在开启事务前就不能对xx进行修改，否则会有异常。在执行事务的时候，会先对监控的数据进行值的比对，然后再去执行事务。

15、【高可用】主从赋值

<https://pdai.tech/md/db/nosql-redis/db-redis-x-copy.html>

说明：

1、关于主从同步选择增量还是全量同步两个关键的缓存区：replication buffer 和 repl_backlog_buffer

Replication Buffer

- **位置：**位于主节点（Master）的内存中。
- **作用：**replication buffer 是用于存储主节点发送给从节点的写命令数据的缓冲区。每当主节点执行写操作时，相关的命令会被存入此缓冲区。
- **特点：**它用于实时传输主节点的写操作给从节点。主节点会不断将该缓冲区中的数据流传输给从节点。这个缓冲区的大小是有限的，因此如果数据量非常大，可能会出现缓冲区溢出，导致一些旧命令被丢弃。

Repl Backlog Buffer

- **位置：**同样位于主节点的内存中。
- **作用：**repl_backlog_buffer 是一个历史命令缓冲区，它用于存储一段时间内的写命令（增量数据）。这个缓冲区会在从节点重新连接时使用，以便从节点能够恢复未接收到的命令，从而追赶上主节点。
- **特点：**它用于处理从节点断开连接或者滞后的情况。当从节点重新连接时，主节点会把 repl_backlog_buffer 中的命令传输给从节点，确保从节点同步主节点的所有增量更新。

区别：

- `replication buffer` 是实时缓存当前正在发送的写命令，用于持续同步主节点和从节点之间的实时数据。
- `repl_backlog_buffer` 是一个历史缓存，存储最近一段时间的写命令，用于帮助从节点恢复丢失的数据，或者弥补从节点与主节点之间的延迟。

总结：

这两个缓冲区都是主节点（Master）上用于确保数据同步的关键部分，分别处理实时命令和历史命令。

`replication buffer` 存储当前需要发送的命令，而 `repl_backlog_buffer` 存储最近一段时间的增量数据，用于从节点连接恢复时的补充同步。

2、关于Redis主线程fork一个子线程去进行RDB创建

如果从库数量很多，而且都要和主库进行全量复制的话，就会导致主库忙于 fork 子进程生成 RDB 文件，进行数据全量复制。fork 这个操作会阻塞主线程处理正常请求，从而导致主库响应应用程序的请求速度变慢。

Redis 的 **主线程** 主要用于处理 **I/O 操作** 和 **key-value 数据的读取/写入**，而它并不会直接参与后台任务的执行。不过，在涉及到 fork 操作时，主线程依然是发起 `fork()` 的地方，但具体的任务执行会由子进程来完成。当涉及到 **持久化操作**（如 RDB 快照、AOF 重写）时，Redis 会启动一个 **子进程** 来执行这些任务（`bgsave`：fork 出一个子进程，子进程执行，不会阻塞 Redis 主线程，默认选项）。这里的 `fork()` 操作是在主线程的控制下进行的。

1.redis使用fork创建子进程就行rdb或aof重写操作时，fork子进程的过程会阻塞主进程，阻塞时间取决于主进程的内存大小-这里你是否觉得有疑问：fork子进程的时候主进程和子进程是采用共享内存的方式进行内存共享的啊，fork阻塞的时间长短怎么会和主进程的内存大小相关呢？

原因是因为主进程fork子进程的时候，子进程需要创建内存页表，一般是几十M左右，主进程的内存越大，内存页表页就越多，所以fork子进程就会越慢。

参考文章：<https://blog.csdn.net/lixia0417mul2/article/details/119922991>

主线程的作用：

- **监听客户端请求**：主线程通过 I/O 多路复用机制，监听所有客户端的连接。当客户端发送请求时，主线程会接收到相应的事件并进行处理。
- **事件处理**：一旦收到事件，主线程会根据事件的类型执行相应的操作，比如读取客户端发送的数据，解析命令，执行相应的 Redis 操作（如读取或修改数据），并将结果返回给客户端。
- **执行非阻塞任务**：由于 Redis 的设计是单线程的，它通过非阻塞的方式执行任务。对于每个客户端的请求，主线程会尽量避免长时间的阻塞操作，确保 Redis 能继续处理其他客户端的请求。

3、psync 命令包含了主库的 runID 和复制进度 offset 两个参数

`PSYNC` 命令是 Redis 中用于 **断点续传复制** 的命令，它用于从主节点恢复复制进度。该命令包含两个主要的参数：**runID** 和 **offset**，这两个参数共同帮助从主节点恢复数据同步。

runID

`runID` 是一个唯一标识符，表示主节点的复制身份（每个redis实例都有一个runID）。它是一个 **32 字符的十六进制字符串**，由主节点生成并唯一标识该节点。在 Redis 中，`runID` 在节点重启时会改变，因此可以通过该 ID 来区分不同的 Redis 实例。

- 主节点生成并持续更新 `runID`，当从节点请求复制时，会将主节点的 `runID` 返回给从节点。

- 在 `PSYNC` 命令中, `runID` 用来确保从节点连接的是正确的主节点。

offset

`offset` 是主节点在给定时间点上复制的进度, 通常表示为 **日志的字节偏移量**。它指示从节点在主节点上成功接收到的数据的数量。`offset` 使得从节点能够从正确的地方恢复复制过程, 而不需要从头开始复制。

- **正常的复制偏移**: 表示当前已同步的数据位置 (通过写入操作的字节数来计量)。
- **断点续传**: 当从节点重新连接主节点时, `offset` 参数表示它想要从哪个字节位置继续同步数据

PSYNC 命令的工作原理

- 当 **从节点** 连接到主节点时, 它会向主节点发送 `PSYNC` 命令, 并提供自己的 `runID` 和 `offset`。
- 主节点会检查从节点提供的`runID`和`offset`是否匹配当前的复制进度:
 - 如果 `runID` 和 `offset` 仍然有效, 主节点会返回剩余的复制数据, 继续从该 `offset` 位置开始同步。
 - 如果 `runID` 与主节点的当前 `runID` 不匹配, 或者 `offset` 超出了主节点的历史日志范围, 主节点会认为从节点已经失去同步, 返回 `FULLRESYNC` 响应, 要求从节点从头开始进行全量同步。

16、【高可用】哨兵机制

<https://pdai.tech/md/db/nosql-redis/db-redis-x-sentinel.html>

<https://www.cnblogs.com/kismetv/p/9609938.html>

说明:

1、Redis中有一个叫做哨兵机制的, 每一个哨兵实例都会对所有主从实例的连接吗?

是的, 在 Redis 的哨兵 (Sentinel) 机制中, 每个哨兵实例都会与所有主从实例建立连接, 并进行监控。具体来说, 每个哨兵实例都会:

监控所有的主节点 (Master)

- 配置文件中通过`sentinel monitor`指定监控的主节点 (哨兵结点的配置文件中都要指定监控的主节点, 如果有多个主节点, 需要都声明, 后续从节点的信息, 也是通过主节点来获取的)
- 通过 `PING` 命令定期检查主节点的健康状态。
- 记录主节点的运行信息, 如当前主从关系、复制状态等。

监控所有的从节点 (Slave)

- 通过 `INFO` (哨兵实例给主库发送 `INFO` 命令, 主库接受到这个命令后, 就会把从库列表返回给哨兵。接着, 哨兵就可以根据从库列表中的连接信息, 和每个从库建立连接, 并在这个连接上持续地对从库进行监控) 和 `PING` 命令获取从节点的健康状态。
- 记录从节点的同步情况, 比如落后主节点的偏移量。
- 在主节点故障转移时, 协助选举新的主节点并通知其他从节点切换主节点。

监控其他哨兵实例

- 通过 Redis 订阅-发布 (Pub/Sub) 机制发现其他哨兵，并相互交换信息，以便对主从状态达成一致。

因此每个哨兵会同时连接所有的 Master 和 Slave 进行健康检查，也会和其他 Sentinel 进行通信，以达成一致决策。

2、哨兵的单点情况

(1) 只有一个Sentinel，无法进行Failover

如果你的**哨兵集群**只有一个 **Sentinel**，当这个 Sentinel 本身宕机或网络隔离时：

- 它无法探测主节点是否存活，整个监控失效。
- 即使主节点真的挂了，没有其他 Sentinel 参与投票，它也无法触发故障转移 (Failover)。

风险：

- 单点故障 (SPOF, Single Point of Failure)，导致 Redis 不能自动切换主从，应用可能出现读写失败。不管是哨兵实例还是主节点故障，都无法完成主从的切换，无法满足对应的切换条件。

解决方案：

- 部署至少 3 个 Sentinel (奇数个，避免投票僵局)。
- 分布在不同的物理机或容器，防止单台机器故障影响整个监控系统。

(2) 仅有 2 个 Sentinel，可能导致投票僵局

如果你只有 2 个 Sentinel，在选举 Leader 时：

- 如果其中一个 Sentinel 宕机或网络异常，剩下的 Sentinel 无法达成 `quorum` 数量，无法触发 Failover。
- 可能出现 1:1 的投票僵局，导致无法选出 Leader。

风险：

- 哨兵集群无法达成共识，故障转移无法执行，Redis 可能会卡在宕机状态。

解决方案：

- 至少部署 3 个 Sentinel，确保投票能顺利进行 (多数决定)。

(3) 所有 Sentinel 在同一台机器，容易失效

如果所有 Sentinel 都部署在同一台服务器，那么：

- 如果该服务器宕机，所有 Sentinel 都会失效，Redis 失去监控能力。
- 如果网络隔离，该服务器上的所有 Sentinel 可能无法与 Redis 服务器通信，导致错误的 Failover 触发。

风险：

- 单机故障 = Sentinel 全部失效，Redis 可能会进入不可用状态。

解决方案：

- 将 Sentinel 部署在不同的服务器或容器，保证物理隔离。

- 使用 Redis Cluster (Redis Cluster 是 Redis 官方提供的 **分布式集群解决方案**，它支持 数据分片 (Sharding) 和 高可用 (HA) ，能够在多个 Redis 实例之间自动管理数据，并提供主从复制和故障转移功能。) + Sentinel 结合，增强高可用性。

(4) 哨兵数量不足，无法形成 Quorum

Redis Sentinel 需要 `quorum` 个 Sentinel 达成共识才能执行故障转移：

- 如果 `sentinel monitor mymaster <master-ip> <port> 2`，意味着至少 2 个 Sentinel 需要判断主节点故障，才能触发 Failover。
- 如果你的 Sentinel 数量小于 `quorum`，或者大部分 Sentinel 宕机，Failover 将无法进行。

风险：

- Sentinel 无法达成投票共识，Redis 集群会进入不可用状态。

解决方案：

- 确保 Sentinel 数量 $\geq \text{quorum} + 1$ ，避免 Sentinel 宕机时影响投票。
- 推荐最少 3 个 Sentinel，最佳是奇数个（例如 3、5、7）。

为了避免哨兵的单点故障，推荐以下 Sentinel 部署策略：

Sentinel 数量	可行性	推荐理由
1 个	❌ 不推荐	失去监控能力，不能自动故障转移
2 个	❌ 不推荐	可能导致选举僵局，Failover 失败
3 个	✅ 推荐	最少的高可用配置，避免僵局
5 个	✅✅ 更推荐	容错能力更强，多个 Sentinel 宕机仍能正常运行

3、哨兵集群Leader选举

为了避免哨兵的单点情况发生，所以需要有一个哨兵的分布式集群。作为分布式集群，必然涉及共识问题（即选举问题）；同时故障的转移和通知都只需要一个主哨兵节点就可以了（即故障转移的时候，需要一个Leader哨兵）

哨兵选举Leader机制全过程：

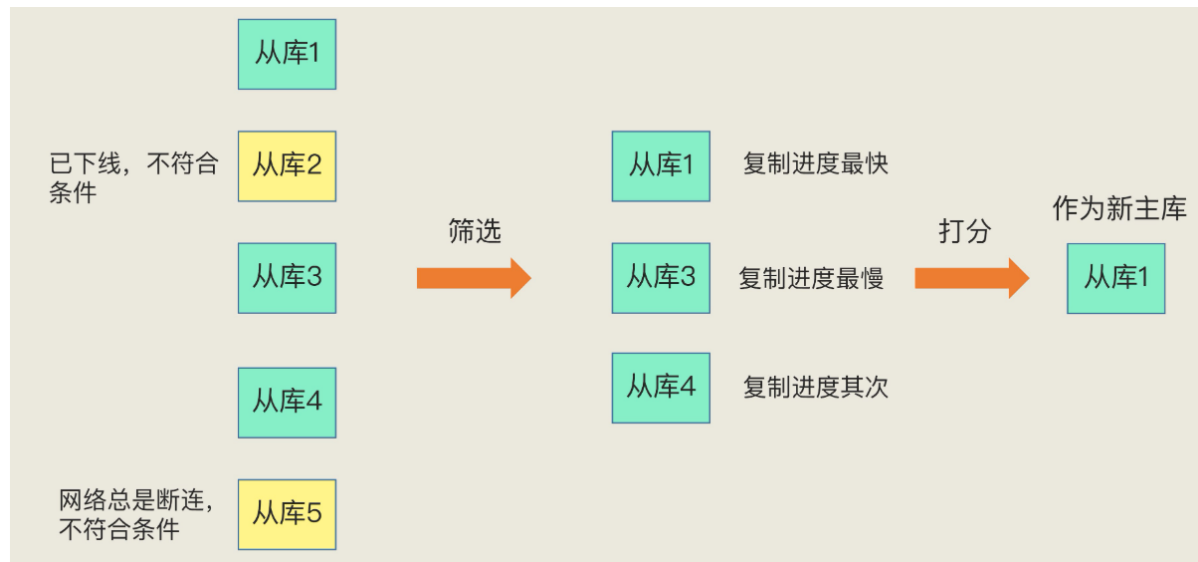
- 检测 Master 故障 (Objective Down, O_DOWN) ，判断客观下线：多个 Sentinel 发现 Master 无法响应 `PING` 请求，标记为 **主观下线 (s_down, Subjectively Down)** 。如果 `quorum` （比如 2 个） Sentinel 都确认 Master 宕机，则认为它 **客观下线 (o_down, Objectively Down)** 。
- Sentinel 竞争成为 Leader：所有存活的 Sentinel 竞争成为 Leader，向其他 Sentinel 发送 `SENTINEL is-master-down-by-addr` 请求，**争取选票。每个 Sentinel 只能投一票**，先达到 **majority (多数投票)** 的 Sentinel 成为 Leader。即通过Raft选举算法进行选举新的主节点，选举的票数大于等于 $\text{num}(\text{sentinels})/2+1$ 时，将成为领导者，如果没有超过，继续选举。任何一个想成为 Leader 的哨兵，要满足两个条件：第一，拿到半数以上的赞成票；第二，拿到的票数同时还需要大于等于哨兵配置文件中的 `quorum` 值
- Leader Sentinel 触发故障转移：选出的 Leader 选择一个 **最合适的 Slave** 作为新的 Master。发送 `SLAVEOF NO ONE` 命令，让这个 Slave 成为新的 Master。让其他 Slave 重新同步新 Master。
- 所有 Sentinel 更新主从关系：**所有 Sentinel 重新配置主从结构**，广播新 Master 信息。Redis 客户端也会收到新的 Master 地址。

以 3 个哨兵为例，假设此时的 quorum 设置为 2，那么，任何一个想成为 Leader 的哨兵只要拿到 2 张赞成票，就可以了。

4、新主库的选出策略

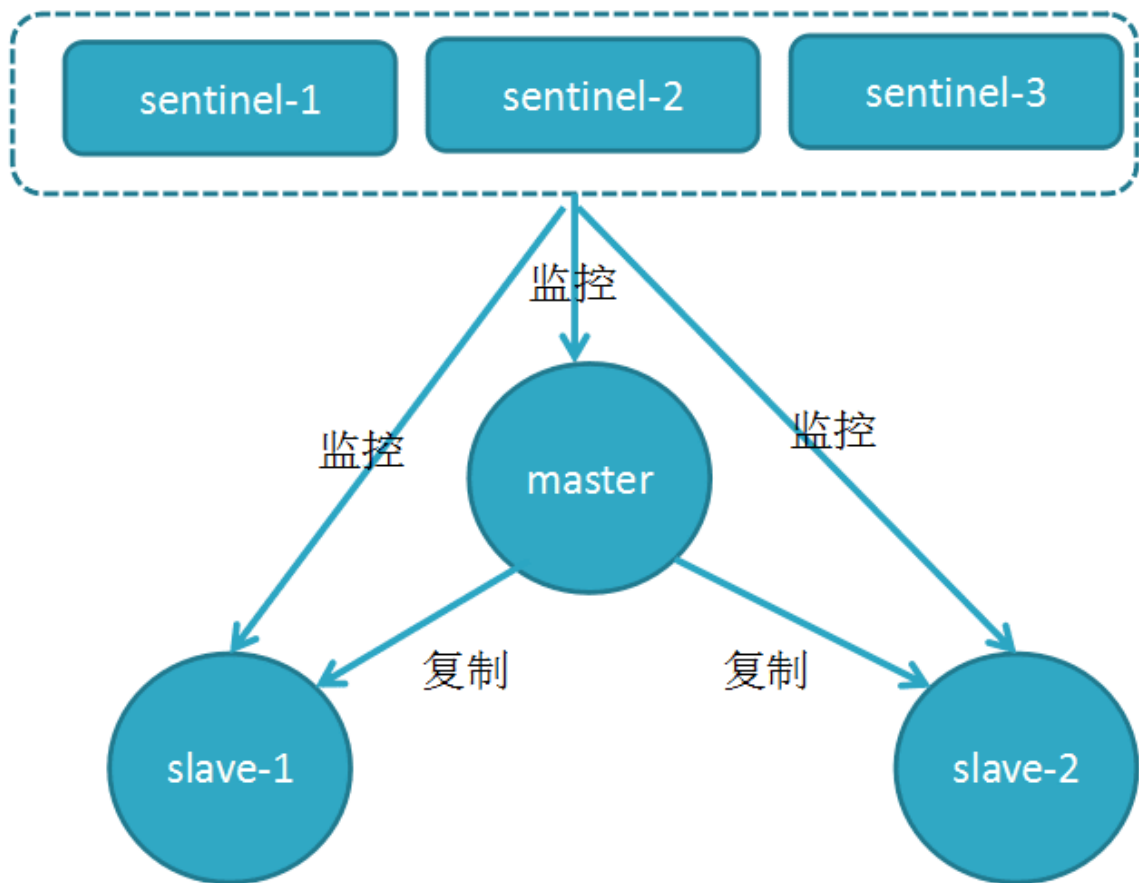
主库判断客观下线后，选出了哨兵leader后，应该如何进行新主库的选择呢？

- 过滤掉不健康的（下线或断线），没有回复过哨兵ping响应的从节点
- 选择 `salve-priority` 从节点优先级最高（redis.conf）的
- 选择复制偏移量最大，只复制最完整的从节点

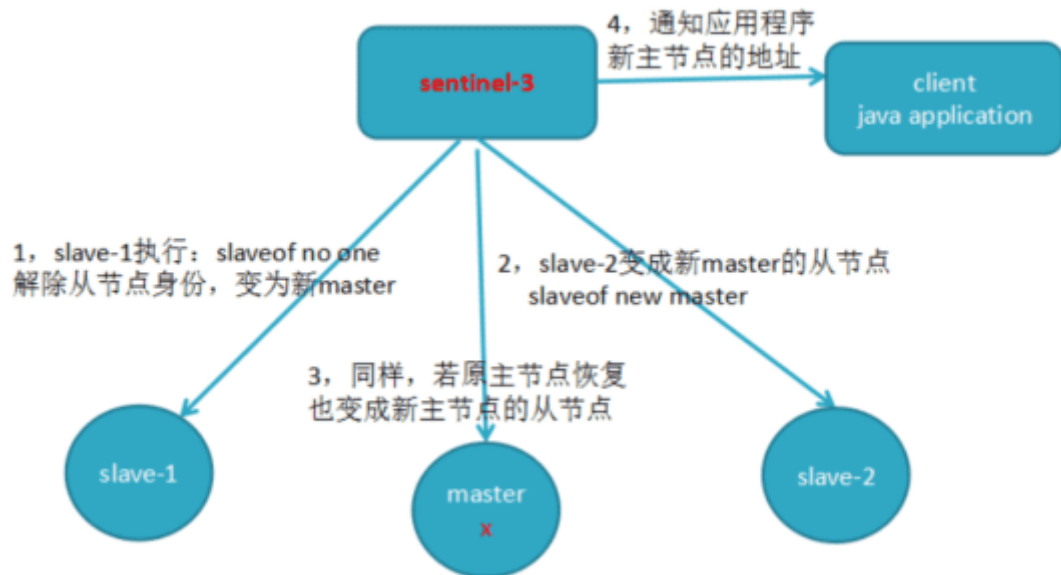


5、故障转移

假设根据我们一开始的图：（我们假设：判断主库客观下线了，同时选出 `sentinel 3` 是哨兵leader）

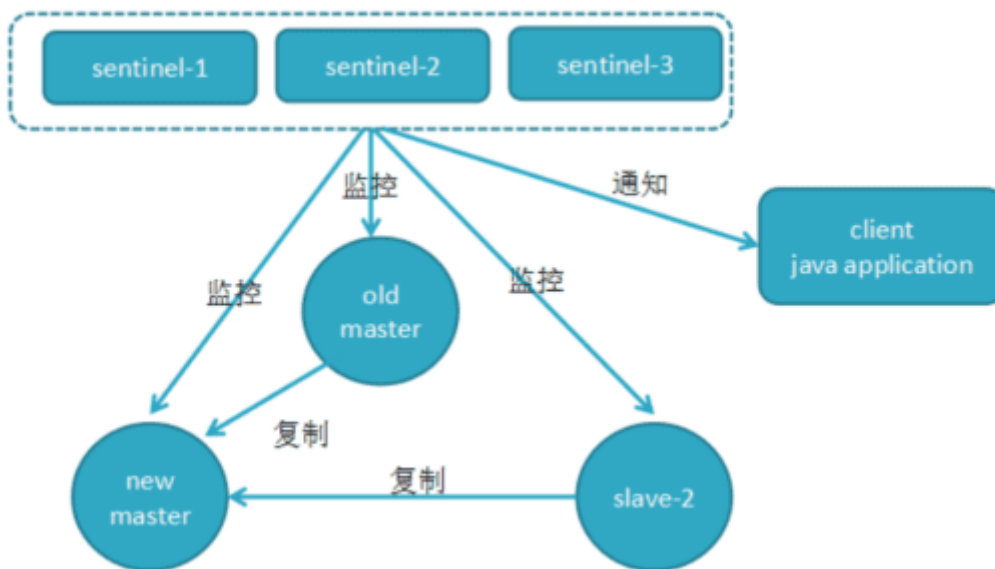


故障转移流程如下：



- 将slave-1脱离原从节点 (PS: 5.0 中应该是 `replicaof no one`), 升级主节点,
- 将从节点slave-2指向新的主节点
- 通知客户端主节点已更换
- 将原主节点 (oldMaster) 变成从节点, 指向新的主节点

转移之后



6、哨兵的功能

Redis Sentinel，即Redis哨兵，在Redis 2.8版本开始引入。**哨兵的核心功能是主节点的自动故障转移。**下面是Redis官方文档对于哨兵功能的描述：

- 监控（Monitoring）：哨兵会不断地检查主节点和从节点是否运作正常。
- 自动故障转移（Automatic failover）：当主节点不能正常工作时，哨兵会自动故障转移操作，它会将失效主节点的其中一个从节点升级为新的主节点，并让其他从节点改为复制新的主节点。
- 配置提供者（Configuration provider）：客户端在初始化时，通过连接哨兵来获得当前Redis服务的主节点地址。

客户端可以通过哨兵节点+masterName获取主节点信息，在这里哨兵起到的作用就是配置提供者。

需要注意的是，哨兵只是配置提供者，而不是代理。二者的区别在于：如果是配置提供者，客户端在通过哨兵获得主节点信息后，会直接建立到主节点的连接，后续的请求(如set/get)会直接发向主节点；如果是代理，客户端的每一次请求都会发向哨兵，哨兵再通过主节点处理请求。

举一个例子可以很好的理解哨兵的作用是配置提供者，而不是代理。在前面部署的哨兵系统中，将哨兵节点的配置文件进行如下修改：

```
sentinel monitor mymaster 192.168.92.128 6379 2
改为
sentinel monitor mymaster 127.0.0.1 6379 2
```

然后，将前述客户端代码在局域网的另外一台机器上运行，会发现客户端无法连接主节点；这是因为哨兵作为配置提供者，客户端通过它查询到主节点的地址为127.0.0.1:6379，客户端会向127.0.0.1:6379建立redis连接，自然无法连接。如果哨兵是代理，这个问题就不会出现了。

- 通知（Notification）：哨兵可以将故障转移的结果发送给客户端。哨兵节点在故障转移完成后，会将新的主节点信息发送给客户端，以便客户端及时切换主节点。具体做法是：利用redis提供的发布订阅功能，为每一个哨兵节点开启一个单独的线程，订阅哨兵节点的+switch-master频道，当收到消息时，重新初始化连接池。

其中，监控和自动故障转移功能，使得哨兵可以及时发现主节点故障并完成转移；而配置提供者和通知功能，则需要在与客户端的交互中才能体现。

这里对“客户端”一词在文章中的用法做一个说明：在前面的文章中，只要通过API访问redis服务器，都会称作客户端，包括redis-cli、Java客户端Jedis等；为了便于区分说明，本文中的客户端并不包括redis-cli，而是比redis-cli更加复杂：redis-cli使用的是redis提供的底层接口，而客户端则对这些接口、功能进行了封装，以便充分利用哨兵的配置提供者和通知功能。

7、哨兵的基本原理（差不多就是运行过程总结）

于哨兵的原理，关键是了解以下几个概念。

（1）定时任务：每个哨兵节点维护了3个定时任务。定时任务的功能分别如下：通过向主从节点发送info命令获取最新的主从结构；通过发布订阅功能获取其他哨兵节点的信息；通过向其他节点发送ping命令进行心跳检测，判断是否下线。

（2）主观下线：在心跳检测的定时任务中，如果其他节点超过一定时间没有回复，哨兵节点就会将其进行主观下线。顾名思义，主观下线的意思是一个哨兵节点“主观地”判断下线；与主观下线相对应的是客观下线。

（3）客观下线：哨兵节点在对主节点进行主观下线后，会通过sentinel is-master-down-by-addr命令询问其他哨兵节点该主节点的状态；如果判断主节点下线的哨兵数量达到一定数值，则对该主节点进行客观下线。

需要特别注意的是，客观下线是主节点才有的概念；如果从节点和哨兵节点发生故障，被哨兵主观下线后，不会再有后续的客观下线和故障转移操作。

（4）选举领导者哨兵节点：当主节点被判断客观下线以后，各个哨兵节点会进行协商，选举出一个领导者哨兵节点，并由该领导者节点对其进行故障转移操作。

监视该主节点的所有哨兵都有可能被选为领导者，选举使用的算法是Raft算法；Raft算法的基本思路是先到先得：即在一轮选举中，哨兵A向B发送成为领导者的申请，如果B没有同意过其他哨兵，则会同意A成为领导者。选举的具体过程这里不做详细描述，一般来说，哨兵选择的过程很快，谁先完成客观下线，一般就能成为领导者。

（5）故障转移：选举出的领导者哨兵，开始进行故障转移操作，该操作大体可以分为3个步骤：

- 在从节点中选择新的主节点：选择的原理是，首先过滤掉不健康的从节点；然后选择优先级最高的从节点(由slave-priority指定)；如果优先级无法区分，则选择复制偏移量最大的从节点；如果仍无法区分，则选择runid最小的从节点。
- 更新主从状态：通过slaveof no one命令，让选出来的从节点成为主节点；并通过slaveof命令让其他节点成为其从节点。
- 将已经下线的主节点(即6379)设置为新的主节点的从节点，当6379重新上线后，它会成为新的主节点的从节点。

通过上述几个关键概念，可以基本了解哨兵的工作原理。为了更形象的说明，下图展示了领导者哨兵节点的日志，包括从节点启动到完成故障转移。

```
66234:X 06 Sep 22:12:59.792 # Sentinel runid is 872cf14ea1c078e7b1a4e1865a9c9f7655f583be
66234:X 06 Sep 22:12:59.792 # +monitor master mymaster 192.168.92.128 6379 quorum 2
66234:X 06 Sep 22:13:00.793 * +slave slave 192.168.92.128:6380 192.168.92.128 6380 @ mymaster 192.168.92.128 6379
66234:X 06 Sep 22:13:00.796 * +slave slave 192.168.92.128:6381 192.168.92.128 6381 @ mymaster 192.168.92.128 6379
66234:X 06 Sep 22:13:01.801 * +sentinel sentinel 192.168.92.128:26379 192.168.92.128 26379 @ mymaster 192.168.92.128 6379
66234:X 06 Sep 22:13:04.180 * +sentinel sentinel 192.168.92.128:26381 192.168.92.128 26381 @ mymaster 192.168.92.128 6379
66234:X 07 Sep 01:26:06.226 # +sdown master mymaster 192.168.92.128 6379
66234:X 07 Sep 01:26:06.299 # +odown master mymaster 192.168.92.128 6379 #quorum 2/2
66234:X 07 Sep 01:26:06.299 # +new-epoch 1
66234:X 07 Sep 01:26:06.299 # +try-failover master mymaster 192.168.92.128 6379
66234:X 07 Sep 01:26:06.301 # +vote-for-leader 872cf14ea1c078e7b1a4e1865a9c9f7655f583be 1
66234:X 07 Sep 01:26:06.306 # 192.168.92.128:26381 voted for 872cf14ea1c078e7b1a4e1865a9c9f7655f583be 1
66234:X 07 Sep 01:26:06.307 # 192.168.92.128:26379 voted for 872cf14ea1c078e7b1a4e1865a9c9f7655f583be 1
66234:X 07 Sep 01:26:06.379 # +elected-leader master mymaster 192.168.92.128 6379
66234:X 07 Sep 01:26:06.379 # +failover-state-select-slave master mymaster 192.168.92.128 6379
66234:X 07 Sep 01:26:06.432 # +selected-slave slave 192.168.92.128:6380 192.168.92.128 6380 @ mymaster 192.168.92.128 6379
66234:X 07 Sep 01:26:06.432 * +failover-state-send-slaveof-noone slave 192.168.92.128:6380 192.168.92.128 6380 @ mymaster 192.168.92.128 6379
66234:X 07 Sep 01:26:06.509 # +failover-state-wait-promotion slave 192.168.92.128:6380 192.168.92.128 6380 @ mymaster 192.168.92.128 6379
66234:X 07 Sep 01:26:06.646 # +promoted-slave slave 192.168.92.128:6380 192.168.92.128 6380 @ mymaster 192.168.92.128 6379
66234:X 07 Sep 01:26:06.646 # +failover-state-reconf-slaves master mymaster 192.168.92.128 6379
66234:X 07 Sep 01:26:06.703 * +slave-reconf-sent slave 192.168.92.128:6381 192.168.92.128 6381 @ mymaster 192.168.92.128 6379
66234:X 07 Sep 01:26:07.396 # -odown master mymaster 192.168.92.128 6379
66234:X 07 Sep 01:26:07.673 * +slave-reconf-inprog slave 192.168.92.128:6381 192.168.92.128 6381 @ mymaster 192.168.92.128 6379
66234:X 07 Sep 01:26:07.673 * +slave-reconf-done slave 192.168.92.128:6381 192.168.92.128 6381 @ mymaster 192.168.92.128 6379
66234:X 07 Sep 01:26:07.729 # +failover-end master mymaster 192.168.92.128 6379
66234:X 07 Sep 01:26:07.729 # +switch-master mymaster 192.168.92.128 6379 192.168.92.128 6380
66234:X 07 Sep 01:26:07.729 * +slave slave 192.168.92.128:6381 192.168.92.128 6381 @ mymaster 192.168.92.128 6380
66234:X 07 Sep 01:26:07.729 * +slave slave 192.168.92.128:6379 192.168.92.128 6379 @ mymaster 192.168.92.128 6380
66234:X 07 Sep 01:26:37.750 # +sdown slave 192.168.92.128:6379 192.168.92.128 6379 @ mymaster 192.168.92.128 6380
66234:X 07 Sep 01:36:04.663 # -sdown slave 192.168.92.128:6379 192.168.92.128 6379 @ mymaster 192.168.92.128 6380
```

1、哨兵节点启动，监控主节点，并发现了从节点和其他哨兵节点

2、对主节点进行主观下线和客观下线

3、进入新的纪元1，并在领导者选举过程中被选为领导者

4、完成故障转移

5、6379重启后，成为从节点

8、客户端原理

<https://www.cnblogs.com/kismetvp/p/9609938.html>

2. 客户端原理

Jedis客户端对哨兵提供了很好的支持。如上述代码所示，我们只需要向Jedis提供哨兵节点集合和masterName，构造JedisSentinelPool对象；然后便可以像使用普通redis连接池一样来使用了：通过pool.getResource()获取连接，执行具体的命令。

在整个过程中，我们的代码不需要显式的指定主节点的地址，就可以连接到主节点；代码中对故障转移没有任何体现，就可以在哨兵完成故障转移后自动的切换主节点。之所以可以做到这一点，是因为在JedisSentinelPool的构造器中，进行了相关的工作；主要包括以下两点：

(1) **遍历哨兵节点，获取主节点信息**：遍历哨兵节点，通过其中一个哨兵节点+masterName获得主节点的信息；该功能是通过调用哨兵节点的sentinel get-master-addr-by-name命令实现，该命令示例如下：

```
root@ubuntu:~# redis-cli -h 192.168.92.128 -p 26379
192.168.92.128:26379> sentinel get-master-addr-by-name mymaster
1) "192.168.92.128"
2) "6380"
```

一旦获得主节点信息，停止遍历（因此一般来说遍历到第一个哨兵节点，循环就停止了）。

(2) **增加对哨兵的监听**：这样当发生故障转移时，客户端便可以收到哨兵的通知，从而完成主节点的切换。具体做法是：利用redis提供的发布订阅功能，为每一个哨兵节点开启一个单独的线程，订阅哨兵节点的+switch-master频道，当收到消息时，重新初始化连接池。

a、对于客观下线和哨兵能够就进行主从切换（用到选举机制）的概念区分：

Redis 1主4从，5个哨兵，哨兵配置quorum为2，如果3个哨兵故障，当主库宕机时，哨兵能否判断主库“客观下线”？能否自动切换？

经过实际测试：

1、哨兵集群可以判定主库“主观下线”。由于quorum=2，所以当有一个哨兵判断主库“主观下线”后，询问另外一个哨兵后也会得到同样的结果，2个哨兵都判定“主观下线”，达到了quorum的值，因此，哨兵集群可以判定主库为“客观下线”。

2、但哨兵不能完成主从切换。哨兵标记主库“客观下线后”，在选举“哨兵领导者”时，一个哨兵必须拿到超过多数的选票(5/2+1=3票)。但目前只有2个哨兵活着，无论怎么投票，一个哨兵最多只能拿到2票，永远无法达到 $N/2+1$ 选票的结果。

b、关于哨兵集群中如何区分主哨兵和从哨兵

在 Redis **哨兵集群 (Sentinel Cluster)** 中，所有哨兵节点 (Sentinel) 是平等的，没有固定的“主哨兵”和“从哨兵”之分。但在**故障转移 (Failover)** 过程中，会临时选举一个 Sentinel 作为 **Leader** 来执行主从切换。

在 Sentinel 机制中，每个 Sentinel 负责：

- **监控 (Monitor)**：持续检测 Redis 主节点 (Master) 和从节点 (Slave) 的健康状态。
- **通知 (Notification)**：如果发现主节点故障，通知其他 Sentinel 和客户端。
- **自动故障转移 (Failover)**：
 - 当 Master 宕机，Sentinel 需要协调选举新的 Master。
 - 选举一个 Sentinel 作为**Leader**，由它执行主从切换。
 - 其他 Sentinel 更新新 Master 信息。

所以，**Sentinel 没有固定的主从之分**，但在故障转移过程中，会**选出一个 Sentinel Leader** 来执行 Failover。

c、Sentinel Leader 不是固定的

- Sentinel Leader 仅在故障转移时选出，平时所有 Sentinel 角色相同。
- 下次故障转移时，可能会选出不同的 Sentinel 作为 Leader。
- 如果当前 Leader 宕机，其他 Sentinel 会重新选举新的 Leader。

17、【高可用扩展】分片技术 (Redis Cluster) (△)

<https://pdai.tech/md/db/nosql-redis/db-redis-x-cluster.html>

有点复杂，先放一放.....

1、主从+哨兵缺点 (为什么需要分片技术)

主从复制和**哨兵机制**保障了高可用，就读写分离而言虽然slave节点扩展了主从的读并发能力，但是**写能力**和**存储能力**是无法进行扩展，就只能是master节点能够承载的上限。

如果面对海量数据那么必然需要构建master (主节点分片)之间的集群，同时必然需要吸收高可用 (主从复制和哨兵机制) 能力，即每个master分片节点还需要有slave节点，这是分布式系统中典型的纵向扩展 (集群的分片技术) 的体现；所以在Redis 3.0版本中对应的设计就是Redis Cluster。

18、缓存问题：一致性, 穿击, 穿透, 雪崩, 污染

<https://pdai.tech/md/db/nosql-redis/db-redis-x-cache.html>

19、版本特性: Redis4.0、5.0、6.0特性整理

<https://pdai.tech/md/db/nosql-redis/db-redis-x-version-latest.html>

20、监控运维

<https://pdai.tech/md/db/nosql-redis/db-redis-y-monitor.html>

21、redis性能调优

<https://pdai.tech/md/db/nosql-redis/db-redis-x-performance.html>

业务api调用慢后排查步骤:

- 在排查前, 首先要对服务器上的redis进行基准性能进行测试, 因为有一个标准, 才可以去对比判断是不是redis真的变慢了?

运行如下命令

```
redis-cli -h 127.0.0.1 -p 6379 --intrinsic-latency 60
```

使用 Redis 的 `--intrinsic-latency` 参数来测试 Redis 实例的内在延迟。在这个例子中, 设置了延迟测试的次数为 `60`, 也就是说, Redis 会执行 60 次请求并记录它们的延迟数据。

`--intrinsic-latency` 是 Redis 命令行客户端 (`redis-cli`) 中的一个参数, 用于测量 Redis 的内在延迟 (也称为 "ping 延迟")。这个参数帮助你测试 Redis 在没有任何外部因素 (如网络延迟或繁忙的服务器负载) 干扰下的基本操作延迟。

当你使用 `--intrinsic-latency` 参数时, Redis 会连续发送一系列的 `PING` 命令到 Redis 服务器, 并记录每个请求的响应时间。这些 `PING` 命令是最基础的命令, 用于测试服务器的最小响应延迟。

输出结果如下所示

```
$ redis-cli -h 127.0.0.1 -p 6379 --intrinsic-latency 60
Max latency so far: 1 microseconds.
Max latency so far: 15 microseconds.
Max latency so far: 17 microseconds.
Max latency so far: 18 microseconds.
Max latency so far: 31 microseconds.
Max latency so far: 32 microseconds.
Max latency so far: 59 microseconds.
Max latency so far: 72 microseconds.

1428669267 total runs (avg latency: 0.0420 microseconds / 42.00 nanoseconds
per run).
worst run took 1429x longer than the average latency.
```

1. **Max latency so far: X microseconds.** 这表示 Redis 在每次测试中的最大延迟。这个延迟是指单次操作的最大时间, 例如:

- `Max latency so far: 1 microseconds` 表示在某次测试中, Redis 操作的最大延迟为 1 微秒。
- 随着测试的进行, 最大延迟逐渐增加, 例如从 1 微秒增长到 72 微秒。

每次新的最大延迟出现时, 它会在控制台上更新。

2. **1428669267 total runs (avg latency: 0.0420 microseconds / 42.00 nanoseconds per run).**

- 1428669267 total runs:** 表示一共进行了 1428669267 次操作。

进行的每个请求只发送一次 PING 到 Redis 服务器，并等待响应,设置的次数（如 60）只是一个基础参数，实际运行时，Redis 会进行更多次的请求来计算延迟的平均值、最大值和最小值等统计数据,以确保测量的可靠性。

- **avg latency: 0.0420 microseconds:** 表示所有测试操作的平均延迟时间为 0.0420 微秒（即 42 纳秒）。这是对所有测试操作延迟的平均值。
- **42.00 nanoseconds per run:** 每次测试操作的平均延迟时间是 42 纳秒。

3. Worst run took 1429x longer than the average latency.

- 这个信息表示最差的测试操作的延迟比平均延迟要长 1429 倍。
- 例如，如果平均延迟是 42 纳秒，那么最差的延迟是 42 纳秒 × 1429 ≈ 60 微秒。这说明某个操作的延迟比大多数操作都要高出很多，通常可能是由于瞬时的网络问题或服务器负载导致。

- 首先查看业务api的调用链路，定位是业务api慢、还是调用mysql的api慢，还是调用redis的api慢

二、补充

1、redis开启事务方式

以为和java集成mysql的（java jdbc）一样，首先要声明语句，然后调用执行方法.....

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;

public class NativeJdbcExample {

    public static void main(String[] args) {
        // 数据库连接的 URL，用户名和密码
        String url = "jdbc:mysql://localhost:3306/yourdb";
        String user = "yourusername";
        String password = "yourpassword";

        // SQL 查询语句
        String sql = "SELECT COUNT(*) FROM users";

        try (Connection connection = DriverManager.getConnection(url, user,
password);
            PreparedStatement stmt = connection.prepareStatement(sql);
            ResultSet rs = stmt.executeQuery()) {
```

```

        if (rs.next()) {
            int count = rs.getInt(1);
            System.out.println("Total number of users: " + count);
        }

    } catch (Exception e) {
        e.printStackTrace();
    }
}
}

```

然后写出了如下代码

```

@PostConstruct
public void init() {
    System.out.println(redisTemplate);
    redisTemplate.opsForValue().set("name", "张三");
    redisTemplate.exec();
}

```

其实调用set的时候就已经执行了，exec是用于redis的事务管理中的提交事务的，在执行exec，需要通过multi开启事务

```

@PostConstruct
public void init() {
    System.out.println(redisTemplate); // 打印 redisTemplate 以调试

    // 启动一个 Redis 事务
    redisTemplate.multi(); // 等同于 Redis 中的 "MULTI"

    // 在事务中排队命令
    redisTemplate.opsForValue().set("name", "张三");

    // 执行事务中的所有命令
    List<Object> execResult = redisTemplate.exec(); // 提交事务
    System.out.println(execResult);
}

```

同样我们也可以通过 `RedisTransactionManager` 配置的方法，配置事务

如果你不想手动管理 `MULTI` 和 `EXEC`，可以使用 Spring 提供的 `RedisTransactionManager` 来管理事务，它会帮你自动处理事务的生命周期。示例如下：

```

@Configuration
public class RedisConfig {

    @Bean
    public RedisTransactionManager redisTransactionManager(RedisTemplate<String,
String> redisTemplate) {
        return new RedisTransactionManager(redisTemplate.getConnectionFactory());
    }
}

```

```
@PostConstruct
public void init() {
    System.out.println(redisTemplate); // 打印 redisTemplate 以调试

    // 使用 RedisTransactionManager 自动管理事务
    redisTemplate.opsForValue().set("name", "张三");
}
```

2、Linux中Docker部署redis的连接

如果 Redis 容器暴露了端口（例如在 Docker run 中使用了 `-p 6379:6379`），你可以直接通过主机的 IP 地址和端口进行连接

```
redis-cli -h <主机IP地址> -p 6379
```

如果 Redis 是在 Docker 内部运行且没有暴露端口，可以使用 Docker 的 `exec` 命令进入 Redis 容器，然后直接使用 Redis 客户端连接：

```
docker exec -it <redis容器ID或容器名称> redis-cli
```

关于docker exec -it 命令的说明：

- `docker exec` 是 Docker 提供的一个命令，用于在运行中的容器内执行命令。它可以让你在不进入容器的情况下，直接在容器内部执行命令。
- `-i` 代表 **interactive**（交互模式），让你在容器中执行命令时保持输入流打开。没有 `-i` 的话，容器内的命令执行会被中断，无法交互。
- `-t` 代表 **tty**（伪终端），它会为你分配一个伪终端，这样你就可以得到一个交互式的 shell 环境。没有 `-t`，即使你进入容器，终端也不会有正常的显示效果

完整形式通常是这样的：

```
docker exec -it <容器名称或容器ID> <命令>
```

例如，如果你想进入一个名为 `my-redis-container` 的容器，并使用 `redis-cli` 客户端，你可以这样执行：

```
docker exec -it my-redis-container redis-cli
```

3、套接字

- https://blog.csdn.net/qq_42606136/article/details/115863353
- <https://www.cnblogs.com/myitnews/p/13790067.html>

发送方的发送数据的处理流程大致为：用户空间 -> 内核 -> 网卡 -> 网络

在用户态空间，调用发送数据接口 send/sendto/write 等写数据包，在内核空间会根据不同的协议走不同的流程。以TCP为例，TCP是一种流协议，内核只是将数据包追加到套接字的发送队列中，真正发送数据的时刻，则是由TCP协议来控制的。TCP协议处理完成之后会交给IP协议继续处理，最后会调用网卡的发送函数，将数据包发送到网卡。

接收方的接收数据的处理流程大致为：网络 -> 网卡 -> 内核(epoll等) -> 进程(业务处理逻辑)

网卡会通过轮询或通知的方式接收数据，Linux做了优化，组合了通知和轮询的机制，简单来说，在CPU响应网卡中断时，不再仅仅是处理一个数据包就退出，而是使用轮询的方式继续尝试处理新数据包，直到没有新数据包到来，或者达到设置的一次中断最多处理的数据包个数。数据离开网卡驱动之后就进入到了协议栈，经过IP层、网络层协议的处理，就会触发IO读事件，比如epoll的reactor模型中，就会触发对应的读事件，然后回调对应的IO处理函数，数据之后会交给业务线程来处理，比如Netty的数据接收处理流程就是这样的。

三、实践

1、可以实践一下：通过Redis5.0的Stream实现消息队列(△)

关于List和Stream都可以实现消息队列的功能

使用Redis实现消息队列时，选择List和Stream的确有不同的优缺点。两者都能有效地处理消息队列，但具体的选择取决于你的应用场景和需求。以下是它们各自的特点：

1. List

Redis的List是一个简单的双向链表结构，常用的操作有 `LPUSH`、`RPUSH`、`LPOP` 和 `RPOP`，可以用于实现消息队列。

优点：

- **简单易用**：List的操作简单，符合传统的消息队列模型，可以通过 `LPUSH` 添加消息，`RPOP` 或 `LPOP` 消费消息，适用于简单的消息队列应用。
- **支持多个消费者**：多个消费者可以同时从队列中弹出消息，适合并发处理。
- **性能**：Redis的List性能较好，尤其是在队列操作上。

缺点：

- **没有消息确认机制**：List本身不提供消息确认机制。如果消费者在处理消息时崩溃，消息会丢失。
- **不支持消息持久化**：虽然Redis本身支持持久化（RDB和AOF），但List中的消息并没有明确的持久化机制，可能会在Redis重启后丢失。
- **有限的功能**：List缺少消息队列管理的高级功能，如延迟队列、消息重试、消息排序等。

2. Stream

Redis 5.0引入了Stream数据类型，专门用于处理消息流，提供更丰富的功能。

优点：

- **消息确认机制**：Stream支持消费者确认（`XACK`），确保消息被成功处理，可以防止消息丢失。
- **消息持久化**：Stream中的消息会被持久化，可以通过 `XREAD` 等命令来获取和消费消息。

- **消费者组**：Stream支持消费者组（`XGROUP`），允许多个消费者协同工作，每个消费者只会处理未处理的消息，避免重复消费。
- **更强大的功能**：Stream支持消息流的管理，能够存储更多的元数据，并且支持消息排序、流的剪裁等高级功能。
- **支持消费历史消息**：Stream允许消费者读取指定时间点的消息，使得消息历史管理更加灵活。

缺点：

- **复杂性较高**：Stream的操作相对复杂，尤其是在管理消费者组和消息确认方面，可能需要更多的开发和运维工作。
- **性能开销**：与List相比，Stream的操作可能有更高的性能开销，因为它管理更多的元数据。

选择建议：

- **使用List**：
 - 如果你的需求比较简单，消息顺序很重要，但没有太多复杂的功能要求。
 - 如果只需要一个简单的“生产者-消费者”模型，且消息确认和历史记录管理不是关键需求。
- **使用Stream**：
 - 如果需要更强大的消息管理功能，例如消息确认、消费者组、持久化、高可用等。
 - 如果需要避免消息丢失，并确保每条消息被消费且不会重复消费。
 - 如果需要处理更复杂的消息队列场景，比如大规模的消息流处理、消息重试、回溯读取历史消息等。

总体来说，如果你需要一个简单的队列，List可以满足大多数需求。如果你的需求更复杂，特别是涉及到消息确认、多个消费者和持久化等，Stream是一个更好的选择。

2. 整理一下IO模型（△）

Reactor 模式是一种用于处理并发 I/O 操作的设计模式，主要用于事件驱动的异步处理模型中。它允许系统以非阻塞的方式高效地处理多个输入/输出事件，常用于高性能的网络服务器或事件驱动的程序（如 web 服务器、数据库驱动、异步消息处理等）。

Reactor 模式的核心概念：

Reactor 模式的关键思想是将事件的发生和事件的处理分离。具体来说，Reactor 模式有两个主要组成部分：

1. **事件源（Event Source）**：通常是 I/O 操作（如网络连接、文件读写等）。
2. **事件分发器（Event Demultiplexer）**：负责监听所有事件源，等待事件发生，并将事件分发给相应的处理器。通常使用操作系统的多路复用技术，如 `select()`，`poll()`，`epoll()` 等。
3. **事件处理器（Event Handler）**：负责处理每个特定事件的逻辑，比如连接建立、数据读取等。

Reactor 模式的工作流程：

1. **事件源注册**：多个 I/O 事件源（如网络连接）会被注册到事件分发器上，监听某些特定的事件（如可读、可写、异常等）。

2. **事件分发器**：事件分发器使用多路复用技术（如 `select`、`poll`、`epoll`）来监视这些事件源，等待事件发生。
3. **事件触发**：当某个事件源发生了预定的事件（比如客户端发送数据），事件分发器会通知相应的事件处理器。
4. **事件处理**：事件处理器会根据事件类型（如连接建立、数据读取等）进行相应的处理。处理完成后，事件分发器继续等待其他事件。

Reactor 模式的分类：

Reactor 模式有多种实现方式，主要包括以下几种：

1. **单线程 Reactor 模式**：在这种模式下，所有的 I/O 事件都由一个单一的线程来处理。事件分发器和事件处理器共享一个线程，该线程既负责监听事件，也负责处理事件。这种模式适用于事件数量较少，或者 I/O 操作相对轻量的场景。
2. **多线程 Reactor 模式**：在多线程 Reactor 模式中，事件分发器和事件处理器分别由多个线程来处理。事件分发器负责监听所有的 I/O 事件并将事件分发给工作线程，工作线程则处理具体的事件。这种模式可以提高并发性能，适用于高负载场景。
3. **Proactor 模式**（与 Reactor 相对）：在 Proactor 模式中，事件的处理（如读取数据）由操作系统完成，应用程序仅提供事件的回调函数。操作系统完成 I/O 操作后，回调函数会被触发执行。与 Reactor 模式相比，Proactor 模式更加依赖于操作系统的异步 I/O 功能。

Reactor 模式的优缺点：

优点：

- **高效的 I/O 处理**：Reactor 模式能够以非阻塞的方式处理大量并发 I/O 请求，避免了传统的多线程阻塞模型带来的上下文切换和性能问题。
- **事件驱动**：所有事件都通过事件驱动机制来处理，能够高效地管理多个 I/O 资源。
- **灵活性**：可以根据需求调整事件分发器和处理器的数量，支持多种并发处理模式。

缺点：

- **编程复杂性**：与传统的阻塞 I/O 模型相比，Reactor 模式需要开发者处理更多的异步事件和回调逻辑，编程模型相对复杂。
- **性能瓶颈**：虽然 Reactor 模式避免了阻塞，但是在高并发环境下，单线程 Reactor 模式仍然可能成为瓶颈，尤其是事件分发和处理的工作负载过高时。

Reactor 模式的应用场景：

Reactor 模式特别适用于需要处理大量并发连接并且 I/O 操作是瓶颈的场景。例如：

- **网络服务器**（如 HTTP 服务器、代理服务器等）。
- **高并发的事件驱动系统**（如高频交易系统、在线游戏服务器等）。
- **异步事件处理系统**（如消息队列、日志处理系统等）。